Ubuntu + qemu + cmake + 交叉工具链环境

我是用的WSL，可以动态占用内存，就不需要预先分配固定内存，而且可以直接访问 Windows 的 C盘、D盘。

WSL之前数据库的时候下过了。

可以检查一下：

按 `Win + R` 键，输入 `powershell` 或 `cmd`，然后回车。

```
wsl -l -v
```



这样就是装过了，而且得是version2。

安装 Ubuntu 到 D 盘

1. `D盘` 创建 **WSL** 文件夹，并在该文件夹下创建 **Ubuntu-20.04** 文件夹（视版本而定，我不知道为啥20.04老下不下来换了24.04）。
2. 安装：

   首先，查看可用的 WSL 发行版：

```
wsl --list --online
```

   安装：

```
wsl --install -d Ubuntu-20.04
```

   推荐安装 20.04 或 22.04 版本，这两个版本最稳定

   安装完毕后会要求你创建一个新用户，按照提示输入用户名和密码即可，然后按 `Ctrl + D` 退出即可。

3. 导出 **Ubuntu-20.04** 为 `.tar` 文件：

```
wsl --export Ubuntu-20.04 D:\WSL\Ubuntu-20.04\Ubuntu-20.04.tar
```

这会下载 Ubuntu-20.04 并将其导出到 `D:\WSL\Ubuntu-20.04\Ubuntu-20.04.tar`，然后可以执行 `ls D:\WSL\Ubuntu-20.04` 查看 `Ubuntu-20.04.tar` 已经成功导出。

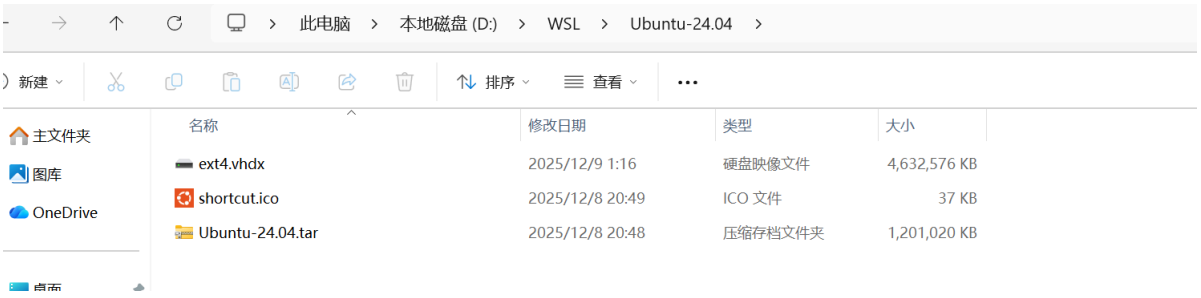
4. 取消注册原有的 Ubuntu-20.04，如果你已经安装了 Ubuntu-20.04（默认在 `C` 盘），可以将其从 WSL 注销：

```
wsl --unregister Ubuntu-20.04
```

5. 导入 Ubuntu-20.04 到 D 盘：

```
wsl --import Ubuntu-20.04 D:\WSL\Ubuntu-20.04 D:\WSL\Ubuntu-20.04\Ubuntu-20.04.tar --version 2
```

这将会把 `Ubuntu-20.04` 安装到 `D` 盘，而不是默认的 `C` 盘



导入完成后，你可以启动 WSL：

```
wsl -d Ubuntu-20.04
```

默认以 `root` 用户登录，也可以创建普通用户

交叉工具链

交叉工具链可以从 Arm 官网的 https://developer.arm.com/downloads/-/arm-gnu-toolchain-downloads 页面下载。

1. 进入家目录

```
cd ~
```

2. Linux：

```
#下载工具链，以下载工具链版本为 11.2，宿主机为 x86 64 位 Linux 机器为例
$ wget https://developer.arm.com/-/media/Files/downloads/gnu/11.2-2022.02/binrel/gcc-arm-11.2-2022.02-x86_64-aarch64-none-elf.tar.xz
```

如果这里下载速度太慢或不动，请用 Windows 浏览器下载这个链接，然后用 `cp /mnt/c/Users/ASUS/Downloads/文件名 .` 把它搬进：https://developer.arm.com/-/media/Files/downloads/gnu/11.2-2022.02/binrel/gcc-arm-11.2-2022.02-x86_64-aarch64-none-elf.tar.xz

执行搬运命令（把文件从 Windows 复制进来）：（注意：命令最后有一个空格和一个点 `.`）

```
cp /mnt/c/Users/ASUS/Downloads/gcc-arm-11.2-2022.02-x86_64-aarch64-none-elf.tar.xz .
```

要注意看文件的下载地址，我是下载到downloads文件夹了，ASUS是我电脑的用户名

可以ls确认一下

3. 解压并重命名

```
tar -xvf gcc-arm-11.2-2022.02-x86_64-aarch64-none-elf.tar.xz
```

重命名文件夹（为了让路径短一点）：

```
mv gcc-arm-11.2-2022.02-x86_64-aarch64-none-elf aarch64-none-elf
```

4. 寻找一下路径：

```
cd ~
ls -d aarch64-none-elf
cd ~/aarch64-none-elf
pwd
```

比如我的是：/home/wenyi/aarch64-none-elf

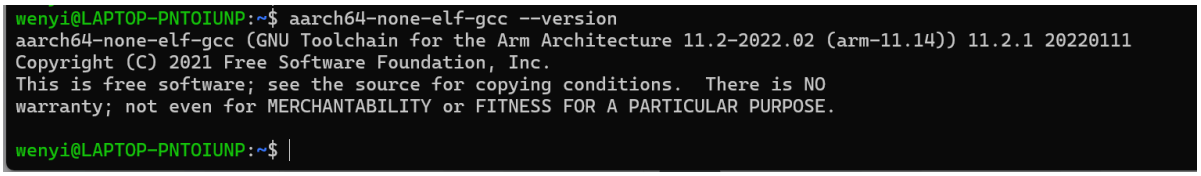
然后返回家目录 cd~

5. 配置环境变量：

```
source ~/.bashrc
vim ~/.bashrc
用 vim 编辑器打开 bash 的配置文件（用来添加环境变量）
```

点i进入插入模式，添加 `export PATH=$PATH:(当前aarch64-none-elf文件所在路径)/aarch64-none-elf/bin`

保存退出后检验一下：



安装qemu

回到家目录

```
sudo apt-get update
sudo apt-get install qemu
sudo apt-get install qemu-system
```

安装CMake

```
sudo apt-get install cmake
```

项目构建

这里不管下哪个项目，都可以用git，或者依旧浏览器下到windows里然后传

类似这种：

```
cp /mnt/d/Downloads/UniProton.tar.gz .
```

后面那个小点千万别忘了

我是下的华为教学的lab9，我给它压缩成zip然后传进ubuntu了

解压：

```
tar -xvf UniProton.tar.gz
```

如果是zip:

```
sudo apt install unzip
unzip UniProton.zip
```

lab9同理。

进入目录编译：

```
cd lab9
# 然后看里面有没有 build 脚本或者 CMakeLists.txt，准备编译
```

可以召唤vscode：

```
code .
```

接下来的编译构建过程是lab9的，它写好了脚本可以一键编译运行：

```
chmod +x *.sh
```

```
./makeMiniEuler.sh
```

其实makeMiniEuler.sh文件长这样：

lab9 > $ makeMiniEuler.sh

```
# sh makeMiniEuler.sh 不打印编译命令
# sh makeMiniEuler.sh -v 打印编译命令等详细信息
rm -rf build/*
mkdir build
cd build
cmake ../src
cmake --build . $1
```

注意：

它这个项目有交叉工具链的绝对位置，记得更改：

lab9 > src > CMakeLists.txt

```
1   cmake_minimum_required(VERSION 3.12)
2
3   set(CMAKE_SYSTEM_NAME "Generic") # 目标系统(baremental):
4   set(CMAKE_SYSTEM_PROCESSOR "aarch64") # 目标系统CPU
5
6   set(TOOLCHAIN_PATH "/usr/local/aarch64-none-elf") # 修改为
7   set(CMAKE_C_COMPILER ${TOOLCHAIN_PATH}/bin/aarch64-none-e
8   set(CMAKE_CXX_COMPILER ${TOOLCHAIN_PATH}/bin/aarch64-none
9   set(CMAKE_ASM_COMPILER ${TOOLCHAIN_PATH}/bin/aarch64-none
10  set(CMAKE_LINKER ${TOOLCHAIN_PATH}/bin/aarch64-none-elf-l
11
```

CMakeLists第六行

然后运行：

```
./runMiniEuler.sh
```

$ runMiniEuler.sh

lab9 > $ runMiniEuler.sh

```
# sh runMiniEuler.sh 直接运行
# sh runMiniEuler.sh -S 启动后在入口处暂停等待调试

echo qemu-system-aarch64 -machine virt -m 1024M -cpu cortex-a53 -nographic -kernel build/miniEuler  -s $1

qemu-system-aarch64 -machine virt -m 1024M -cpu cortex-a53 -nographic -kernel build/miniEuler  -s $1
```